Core Hours

Knowledge Unit	CS Core	KA Core
Foundational Data Structures and Algorithms	11	6
Algorithmic Strategies	6	
Complexity Analysis	6	3
Computational Models and Formal Languages	9	23
Society, Ethics, and the Profession	Included in SEP hours	
Total	**32**	**32**

The 11 CS Core hours in Foundational Data Structures and Algorithms are in addition to 9 hours counted in SDF and 3 hours counted in MSF.

Knowledge Units

AL-Foundational: Foundational Data Structures and Algorithms

CS Core: (See also: SDF-Data-Structures, SDF-Algorithms)

1. Abstract Data Type (ADT) and operations on an ADT (See also: FPL-Types)
   a. Dictionary operations (insert, delete, find)
2. Arrays
   a. Numeric vs non-numeric, character strings
   b. Single (vector) vs multidimensional (matrix)
3. Records/Structs/Tuples and Objects (See also: FPL-OOP)
4. Linked lists (for historical reasons)
   a. Single vs Double and Linear vs Circular
5. Stacks
6. Queues and deques
   a. Heap-based priority queue
7. Hash tables/maps
   a. Collision resolution and complexity (e.g., probing, chaining, rehash)
8. Graphs (e.g., [un]directed, [a]cyclic, [un]connected, and [un]weighted) (See also: MSF-Discrete)
   a. Graph representation: adjacency list vs matrix
9. Trees (See also: MSF-Discrete)
   a. Binary, n-ary, and search trees
   b. Balanced (e.g., AVL, Red-Black, Heap)
10. Sets (See also: MSF-Discrete)

11. Search algorithms
    a. O(n) complexity (e.g., linear/sequential array/list search)
    b. $O(\log_2 n)$ complexity (e.g., binary search)
    c. $O(\log_b n)$ complexity (e.g., uninformed depth/breadth-first tree search)
12. Sorting algorithms (e.g., stable, unstable)
    a. $O(n^2)$ complexity (e.g., insertion, selection),
    b. $O(n \log n)$ complexity (e.g., quicksort, merge, timsort)
13. Graph algorithms
    a. Shortest path (e.g., Dijkstra's, Floyd's)
    b. Minimal spanning tree (e.g., Prim's, Kruskal's)

KA Core:

14. Sorting algorithms
    a. $O(n \log n)$ complexity heapsort
    b. Pseudo $O(n)$ complexity (e.g., bucket, counting, radix)
15. Graph algorithms
    a. Transitive closure (e.g., Warshall's)
    b. Topological sort
16. Matching
    a. Efficient string matching (e.g., Boyer-Moore, Knuth-Morris-Pratt)
    b. Longest common subsequence matching
    c. Regular expression matching

Non-core:

17. Cryptography algorithms (e.g., SHA-256) (See also: SEC-Crypto)
18. Parallel algorithms (See also: PDC-Algorithms, FPL-Parallel)
19. Consensus algorithms (e.g., Blockchain) (See also: SEC-Crypto)
    a. Proof of work vs proof of stake (See also: SEP-Sustainability)
20. Quantum computing algorithms (See also: AL-Models, AR-Quantum)
    a. Oracle-based (e.g., Deutsch-Jozsa, Bernstein-Vazirani, Simon)
    b. Superpolynomial speed-up via QFT (e.g., Shor's)
    c. Polynomial speed-up via amplitude amplification (e.g., Grover's)
21. Fast-Fourier Transform (FFT) algorithm
22. Differential evolution algorithm

Illustrative Learning Outcomes:

CS Core:

1. For each ADT/Data-Structure in this unit
    a. Explain its definition, properties, representation(s), and associated ADT operations.
    b. Explain step-by-step how the ADT operations associated with the data structure transform it.
2. For each algorithm in this unit explain step-by-step how the algorithm operates.
3. For each algorithmic approach (e.g., sorting) in this unit apply a prototypical example of the approach (e.g., merge sort).

4. Given requirements for a problem, develop multiple solutions using various data structures and algorithms. Subsequently, evaluate the suitability, strengths, and weaknesses selecting an approach that best satisfies the requirements.
5. Explain how collision avoidance and collision resolution is handled in hash tables.
6. Explain factors beyond computational efficiency that influence the choice of algorithms, such as programming time, maintainability, and the use of application-specific patterns in the input data.
7. Explain the heap property and the use of heaps as an implementation of a priority queue.

KA Core:

8. For each of the algorithms and algorithmic approaches in the KA Core topics:
    a. Explain a prototypical example of the algorithm, and
    b. Explain step-by-step how the algorithm operates.

Non-core:

9. An appreciation of quantum computation and its application to certain problems.

AL-Strategies: Algorithmic Strategies

CS Core:

1. Paradigms
    a. Brute-Force (e.g., linear search, selection sort, traveling salesperson, knapsack)
    b. Decrease-and-Conquer
        i. By a Constant (e.g., insertion sort, topological sort),
        ii. By a Constant Factor (e.g., binary search),
        iii. By a Variable Size (e.g., Euclid's)
    c. Divide-and-Conquer (e.g., binary search, quicksort, mergesort, Strassen's)
    d. Greedy (e.g., Dijkstra's, Kruskal's, Knapsack)
    e. Transform-and-Conquer
        i. Instance simplification (e.g., find duplicates via list presort)
        ii. Representation change (e.g., heapsort)
        iii. Problem reduction (e.g., least-common-multiple, linear programming)
        iv. Dynamic programming (e.g., Floyd's, Warshall, Bellman-Ford)
    f. Space vs time tradeoffs (e.g., hashing)
2. Handling exponential growth (e.g., heuristic A*, branch-and-bound, backtracking)
3. Iteration vs recursion (e.g., factorial, tree search)

KA Core:

4. Paradigms
    a. Approximation algorithms
    b. Iterative improvement (e.g., Ford-Fulkerson, simplex)
    c. Randomized/Stochastic algorithms (e.g., max-cut, balls and bins)

Non-core:

5. Quantum computing

Illustrative Learning Outcomes:

CS Core:

1. For each of the paradigms in this unit,
   a. Explain its definitional characteristics,
   b. Explain an example that demonstrates the paradigm including how this example satisfies the paradigm's characteristics.
2. For each of the algorithms in the AL-Foundational unit, explain the paradigm used by the algorithm and how it exemplifies this paradigm.
3. Given an algorithm, explain the paradigm used by the algorithm and how it exemplifies this paradigm.
4. Give a real-world problem, evaluate appropriate algorithmic paradigms and algorithms from these paradigms that address the problem including evaluating the tradeoffs among the paradigms and algorithms selected.
5. Give examples of iterative and recursive algorithms that solve the same problem, explain the benefits and disadvantages of each approach.
6. Evaluate whether a greedy approach leads to an optimal solution.
7. Explain various approaches for addressing computational problems whose algorithmic solutions are exponential.

AL-Complexity: Complexity

CS Core:

1. Complexity Analysis Framework
   a. Best, average, and worst-case performance of an algorithm
   b. Empirical and relative (Order of Growth) measurements
   c. Input size and primitive operations
   d. Time and space efficiency
2. Asymptotic complexity analysis (average and worst-case bounds)
   a. Big-O, Big-Omega, and Big-Theta formal notations
   b. Foundational Complexity Classes and Representative Examples/Problems
      i. $O(1)$ *Constant* (e.g., array access)
      ii. $O(\log_2 n)$ *Logarithmic* (e.g., binary search)
      iii. $O(n)$ *Linear* (e.g., linear search)
      iv. $O(n \log_2 n)$ *Log Linear* (e.g., mergesort)
      v. $O(n^2)$ *Quadratic* (e.g., selection sort)
      vi. $O(n^c)$ *Polynomial* (e.g., $O(n^3)$ Gaussian elimination)
      vii. $O(2^n)$ *Exponential* (e.g., Knapsack, Satisfiability (SAT), Traveling Sales-Person (TSP), all subsets)
      viii. $O(n!)$ *Factorial* (e.g., Hamiltonian circuit, all permutations)
3. Empirical measurements of performance
4. Tractability and intractability
   a. P, NP, and NP-Complete Complexity Classes
   b. NP-Complete Problems (e.g., SAT, Knapsack, TSP)
   c. Reductions
5. Time and space tradeoffs in algorithms

KA Core:

6. Little-o, Little-Omega, and Little Theta notations
7. Formal recursive analysis
8. Amortized analysis
9. Turing Machine-based models of complexity
    a. Time complexity
        i. P, NP, NP-C, and EXP classes
        ii. Cook-Levin theorem
    b. Space Complexity
        i. NSpace and PSpace
        ii. Savitch's theorem

Illustrative Learning Outcomes:

CS Core:

1. Prepare a presentation that explains to first year students the basic concepts of algorithmic complexity including best, average, and worst-case algorithm behavior, Big- O, Omega, and Theta notations, complexity classes, time and space tradeoffs, empirical measurement, and impact on practical problems.
2. Using examples, explain each of the foundational complexity classes in this unit.
3. For each foundational complexity class in this unit, explain an algorithm that demonstrates the associated runtime complexity.
4. For each algorithm in the AL-Foundational unit, explain its runtime complexity class and why it belongs to this class.
5. Informally evaluate the foundational complexity class of simple algorithms.
6. Given a problem to program for which there may be several algorithmic approaches, evaluate them and determine which are feasible, and select one that is optimal in implementation and run-time behavior.
7. Develop empirical studies to determine and validate hypotheses about the runtime complexity of various algorithms by running algorithms on input of various sizes and comparing actual performance to the theoretical analysis.
8. Explain examples that illustrate time-space tradeoffs of algorithms.
9. Explain how tree balance affects the efficiency of binary search tree operations.
10. Explain to a non-technical audience the significance of tractable versus intractable algorithms using an intuitive explanation of Big-O complexity.
11. Explain the significance of NP-Completeness.
12. Explain how NP-Hard is a lower bound and NP is an upper bound for NP-Completeness.
13. Explain examples of NP-complete problems.

KA Core:

14. Use recurrence relations to evaluate the time complexity of recursively defined algorithms.
15. Apply elementary recurrence relations using a form of the Master Theorem.
16. Apply Big-O notation to give upper case bounds on time/space complexity of algorithms.
17. Explain the Cook-Levin Theorem and the NP-Completeness of SAT.

18. Explain the classes P and NP.
19. Prove that a problem is NP-Complete by reducing a classic known NP-C problem to it (e.g., 3SAT and Clique).
20. Explain the P-space class and its relation to the EXP class.

AL-Models: Computational Models and Formal Languages

CS Core:

1. Formal automata
   a. Finite State
   b. Pushdown
   c. Linear Bounded
   d. Turing Machine
2. Formal languages, grammars and Chomsky Hierarchy
   (See also: FPL-Translation, FPL-Syntax)
   a. Regular (Type-3)
      i. Regular Expressions
   b. Context-Free (Type-2)
   c. Context-Sensitive (Type-1)
   d. Recursively Enumerable (Type-0)
3. Relations among formal automata, languages, and grammars
4. Decidability, (un)computability, and halting
5. The Church-Turing thesis
6. Algorithmic correctness
   a. Invariants (e.g., in iteration, recursion, tree search)

KA Core:

7. Deterministic and nondeterministic automata
8. Pumping Lemma proofs
   a. Proof of Finite State/Regular-Language limitation
   b. Pushdown Automata/Context-Free-Language limitation
9. Decidability
   a. Arithmetization and diagonalization
10. Reducibility and reductions
11. Time complexity based on Turing Machine
12. Space complexity (e.g., Pspace, Savitch's Theorem)
13. Equivalent models of algorithmic computation
    a. Turing Machines and Variations (e.g., multi-tape, non-deterministic)
    b. Lambda Calculus (See also: FPL-Functional)
    c. Mu-Recursive Functions

Non-core:

14. Quantum computation (See also: AR-Quantum)
    a. Postulates of quantum mechanics
       i. State space

ii. State evolution
iii. State composition
iv. State measurement

b. Column vector representations of qubits
c. Matrix representations of quantum operations
d. Simple quantum gates (e.g., XNOT, CNOT)

Illustrative Learning Outcomes:
CS Core:

1. For each formal automaton in this unit:
   a. Explain its definition comparing its characteristics with this unit's other automata,
   b. Using an example, explain step-by-step how the automaton operates on input including whether it accepts the associated input,
   c. Explain an example of inputs that can and cannot be accepted by the automaton.
2. Given a problem, develop an appropriate automaton that addresses the problem.
3. Develop a regular expression for a given regular language expressed in natural language.
4. Explain the difference between regular expressions (Type-3 acceptors) and the regular expressions (Type-2 acceptors) used in programming languages.
5. For each formal model in this unit:
   a. Explain its definition comparing its characteristics with the others in this unit,
   b. Explain example inputs that are and cannot be accepted by the language/grammar.
6. Explain a universal Turing Machine and its operation.
7. Present to an audience of co-workers and managers the impossibility of providing them a program that checks all other programs, including some seemingly simple ones, for infinite loops including an explanation of the Halting problem, why it has no algorithmic solution, and its significance for real-world algorithmic computation.
8. Explain examples of classic uncomputable problems.
9. Explain the Church-Turing Thesis and its significance for algorithmic computation.
10. Explain how (loop) invariants can be used to prove the correctness of an algorithm.

Illustrative Learning Outcomes:
KA Core:

11. For each formal automaton in this unit explain (compare/contrast) its deterministic and nondeterministic capabilities.
12. Apply pumping lemmas, or alternative means, to prove the limitations of Finite State and Pushdown automata.
13. Apply arithmetization and diagonalization to prove the Halting Problem for Turing Machines is Undecidability.
14. Given a known undecidable language, apply a mapping reduction or computational history to prove that another language is undecidable.
15. Convert among equivalently powerful notations for a language, including among DFAs, NFAs, and regular expressions, and between PDAs and CFGs.
16. Explain Rice's theorem and its significance.

17. Explain an example proof of a problem that is uncomputable by reducing a classic known uncomputable problem to it.
18. Explain the Primitive and General Recursive functions (zero, successor, selection, primitive recursion, composition, and Mu), their significance, and Turing Machine implementations.
19. Explain how computation is performed in Lambda Calculus (e.g., Alpha conversion and Beta reduction)

Non-core:

20. For a quantum system give examples that explain the following postulates.
    a. State Space – system state represented as a unit vector in Hilbert space,
    b. State Evolution – the use of unitary operators to evolve system state,
    c. State Composition – the use of tensor product to compose systems states,
    d. State Measurement – the probabilistic output of measuring a system state.
21. Explain the operation of a quantum XNOT or CNOT gate on a quantum bit represented as a matrix and column vector, respectively.

AL-SEP: Society, Ethics, and the Profession

CS Core: (See also: SEP-Context, SEP-Sustainability)

1. Social, ethical, and secure algorithms
2. Algorithmic fairness
3. Anonymity (e.g., Differential Privacy)
4. Accountability/Transparency
5. Responsible algorithms
6. Economic and other impacts of inefficient algorithms
7. Sustainability

KA Core:

8. Context aware computing

Illustrative Learning Outcomes:

CS Core:

1. Develop algorithmic solutions to real-world societal problems, such as routing an ambulance to a hospital.
2. Explain the impact that an algorithm may have on the environment and society when used to solve a real-world problem while considering its sustainability and that it can affect different societal groups in different ways.
3. Prepare a presentation that justifies the selection of appropriate data structures and/or algorithms to solve a given real-world problem.
4. Explain an example that articulates how differential privacy protects knowledge of an individual's data.
5. Explain the environmental impacts of design choices that relate to algorithm design.
6. Explain the tradeoffs involved in proof-of-work and proof-of-stake algorithms.